T3000 API & Protocol

Centralized Database: Multi-PC Architecture (v4 Final) · built Nov 13, 2026

Centralized Database: Multi-PC Architecture (v4 Final)

> **Status:** Approved design — ready for implementation **Last updated:** 2025-01 **Related docs:**
>
> - centralized-database-design.md — backend config, encryption, schema
> - multipc-data-flow-details.md — per-table data flow reference

1. Problem Statement

T3000 manages building-automation panels (Temco BACnet controllers). A single site may have **multiple PCs** running T3000, each scanning different panels via FFI (T3000.exe DLL). Today each PC writes to its own local SQLite, so:

- No unified view of all panels across PCs.
- Trendlog detail data grows large, bloating local SQLite files.
- No way for a remote browser to see everything in one place.

Goal: Let all PCs share one central database (MSSQL / PostgreSQL / MySQL) while keeping local SQLite as a safe fallback.

2. Scenarios

Scenario A — Single PC (Classic Mode)

TEXT

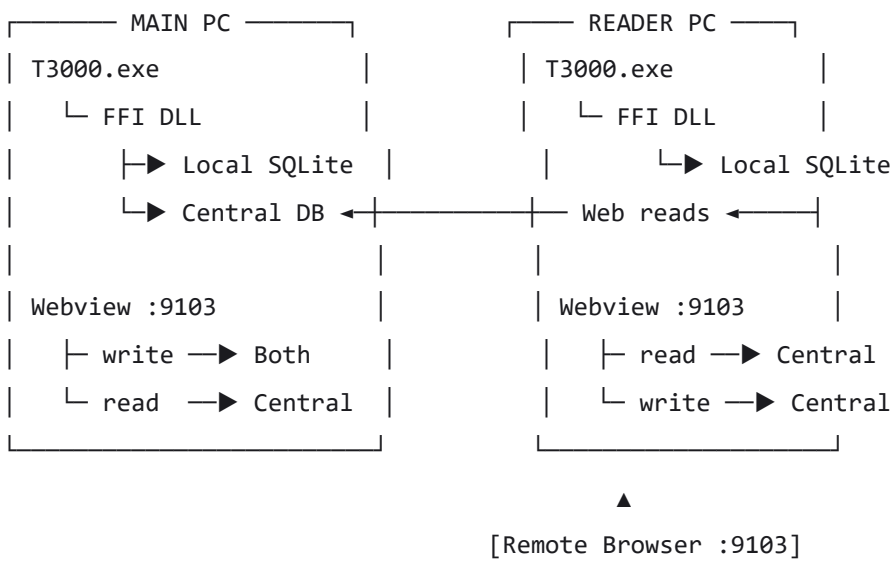
```
[T3000.exe + DLL] ──FFI──▶ [Local SQLite]
                                  |
                      [Webview Server :9103]
                                  |
                          [Browser UI]
```

- `enabled = 0` or no `[CentralDatabase]` section in `setting.ini`.

- Everything works exactly as today. Feature is invisible.

Scenario B — Multi-PC with Central DB

TEXT



Key rules:

1. **Main PC:** FFI writes basic data to **both** local + central. TRENDLOG_DATA_DETAIL goes to central **only**.
2. **Reader PC:** FFI writes basic data to **local only** (for fallback). Web server reads/writes from **central**.
3. **Any browser** connecting to any PC sees the same central data.

---

3. Configuration Design

3.1 setting.ini (per-PC, local file)

CONFIGURATION

```
[CentralDatabase]
enabled=1
role=main          ; "main" or "reader"
store_logs=1       ; write SYSTEM_LOGS to central DB
```

- **Location:** Same directory as T3000.exe (`C:\T3000\setting.ini` typically).
- **Why INI?** Each PC must independently know its role. This cannot be stored in a shared DB because the role determines *how* we talk to that DB.
- **Security:** Cannot be changed via remote web access — only local file edit.

3.2 DB_BACKEND_CONFIG table (local SQLite)

Stores the connection details for the central database:

Column	Purpose
`id`	Primary key
`backend_type`	`"sqlite"`, `"postgres"`, `"mysql"`, `"mssql"`
`host`	Server hostname/IP
`port`	Server port
`database_name`	Database name
`username`	Login username
`password_encrypted`	AES-256-GCM encrypted password
`use_ssl`	SSL flag
`is_active`	Which config is currently active
`role`	Mirrors INI role (for API convenience)
`store_logs`	Mirrors INI store_logs

3.3 APPLICATION_CONFIG table (local SQLite)

Runtime feature flags (key-value):

Key	Value	Purpose
`central_db.enabled`	`"1"` / `"0"`	Feature toggle
`central_db.role`	`"main"` / `"reader"`	PC role
`central_db.store_logs`	`"1"` / `"0"`	Log routing

These are populated at startup from `setting.ini` and `DB_BACKEND_CONFIG`.

4. Complete Table List — All 47 Tables

Category A: FFI-Synced Device Data (basic points)

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
1	DEVICES	Yes	Local + Central	Local only	Central
2	INPUTS	Yes	Local + Central	Local only	Central
3	OUTPUTS	Yes	Local + Central	Local only	Central
4	VARIABLES	Yes	Local + Central	Local only	Central

Category B: FFI-Synced Config/Programming

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
5	PROGRAMS	Yes	Local + Central	Local only	Central
6	SCHEDULES	Yes	Local + Central	Local only	Central
7	PID_TABLE	Yes	Local + Central	Local only	Central
8	HOLIDAYS	Yes	Local + Central	Local only	Central

Category C: Trendlog Metadata

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
9	TRENDLOGS	Yes	Local + Central	Local only	Central
10	TRENDLOG_INPUTS	Yes	Local + Central	Local only	Central
11	TRENDLOG_DATA	Yes	Local + Central	Local only	Central

Category D: Trendlog Detail Data (HIGH VOLUME — central only)

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
12	TRENDLOG_DATA_DETAIL	Yes	**Central ONLY**	Skip (not written)	Central

> **Why central only?** This table grows unbounded (millions of rows). Writing it to local SQLite defeats the purpose of centralization. On reader PCs it is not written at all because those panels are scanned by the main PC.

Category E: UI / Config Tables

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
13	GRAPHICS	No	Local + Central	—	Central
14	GRAPHIC_LABELS	No	Local + Central	—	Central
15	ALARMS	No*	Local + Central	—	Central
16	ALARM_SETTINGS	No	Local + Central	—	Central
17	MONITORDATA	No	Local + Central	—	Central
18	ARRAYS	No	Local + Central	—	Central
19	CONVERSION_TABLES	No	Local + Central	—	Central
20	CUSTOM_UNITS	No	Local + Central	—	Central
21	VARIABLE_UNITS	No	Local + Central	—	Central
22	USERS	No	Local + Central	—	Central
23	REMOTE_POINTS	No	Local + Central	—	Central
24	EMAIL_ALARMS	No	Local + Central	—	Central
25	EXTIO_DEVICES	No	Local + Central	—	Central
26	TSTAT_SCHEDULES	No	Local + Central	—	Central
27	MSV_DATA	No	Local + Central	—	Central

> *ALARMS may be populated by FFI in future; currently web-only. "—" under Reader PC Destination means these tables are managed via the web server, which writes to Central when enabled.

Category F: Device Settings

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
28	NETWORK_SETTINGS	No	Local + Central	—	Central
29	COMMUNICATION_SETTINGS	No	Local + Central	—	Central
30	PROTOCOL_SETTINGS	No	Local + Central	—	Central
31	TIME_SETTINGS	No	Local + Central	—	Central
32	DYNDNS_SETTINGS	No	Local + Central	—	Central
33	HARDWARE_INFO	No	Local + Central	—	Central
34	FEATURE_FLAGS	No	Local + Central	—	Central
35	WIFI_SETTINGS	No	Local + Central	—	Central
36	MISC_SETTINGS	No	Local + Central	—	Central

Category G: Local-Only Infrastructure (NEVER go to central)

#	Table	Purpose	Why Local Only
37	APPLICATION_CONFIG	Feature flags, sync intervals	Per-PC settings
38	APPLICATION_CONFIG_HISTORY	Config change audit trail	Per-PC history
39	DB_BACKEND_CONFIG	Central DB connection details	Contains encrypted credentials
40	database_partition_config	Partition management config	Local DB management
41	database_files	Partition file tracking	Local DB management
42	database_partitions	Partition metadata	Local DB management
43	DATA_SYNC_METADATA	FFI sync cycle tracking	Per-PC sync state
44	TRENDLOG_DATA_SYNC_METADATA	Trendlog sync tracking	Per-PC sync state

Category H: Legacy / Deprecated (LOCAL only)

#	Table	Notes
45	TRENDLOG_DATA_OLD	Legacy format, kept for migration
46	TRENDLOG_VIEWS	UI view configuration
47	REMOTE_TSTAT_DB	Legacy remote thermostat cache

New Table: SYSTEM_LOGS (to be added)

#	Table	Main PC Destination	Reader PC Destination	Web Server Source
48	SYSTEM_LOGS	Central (if store_logs=1)	Central (if store_logs=1)	Central

> SYSTEM_LOGS captures application events, errors, and audit entries. Controlled by `store_logs` flag in setting.ini.

---

5. Startup Flow

TEXT

```
App starts
  │
  ├─ 1. Read setting.ini [CentralDatabase] section
  │     → enabled, role, store_logs
  │     → If section missing or enabled=0 → classic mode, done
  │
  ├─ 2. Open local SQLite (webview_t3_device.db)
  │     → Read DB_BACKEND_CONFIG where is_active=1
  │     → Decrypt password
  │
  ├─ 3. Connect to central DB
  │     → Run schema migration if needed (create tables)
  │     → Store connection in T3AppState
  │
  ├─ 4. Populate T3AppState fields:
  │     → central_db_enabled = true
  │     → central_db_role = "main" or "reader"
  │     → store_logs_to_central = true/false
  │     → t3_device_conn = central DB connection
  │     → local_config_conn = local SQLite (always kept)
  │
  └─ 5. Start services:
        → FFI sync service (behavior varies by role)
        → Web server (reads/writes from central)
```

---

6. FFI Sync Service Behavior

Main PC

RUST

```rust
// For DEVICES, INPUTS, OUTPUTS, VARIABLES, PROGRAMS, SCHEDULES,
// PID_TABLE, HOLIDAYS, TRENDLOGS, TRENDLOG_INPUTS, TRENDLOG_DATA:
fn sync_basic_data(panel_data) {
    write_to_local_sqlite(panel_data);    // always — fallback safety
    write_to_central_db(panel_data);      // dual write
}

// For TRENDLOG_DATA_DETAIL:
fn sync_trendlog_detail(detail_rows) {
    write_to_central_db(detail_rows);     // central ONLY — no local copy
    // Local SQLite is NOT written — saves gigabytes
}
```

Reader PC

RUST

```rust
// FFI still runs — this PC scans its own assigned panels
fn sync_basic_data(panel_data) {
    write_to_local_sqlite(panel_data);    // local only — for fallback
    // Does NOT write to central — main PC owns central writes
}

// TRENDLOG_DATA_DETAIL:
fn sync_trendlog_detail(detail_rows) {
    // Not applicable — reader panels' trendlog detail
    // should be handled by whichever PC is "main" for those panels
    // OR: write to central if this reader also collects trend data
    write_to_central_db(detail_rows);     // optional: central only
}
```

> **Important:** Reader FFI writing to local SQLite ensures that if the central DB goes down, the reader PC can switch back to classic mode and still have recent data.

---

7. Web Server Behavior

When central_db_enabled = true

Operation	Connection Used
Read any table (A–F, H)	Central DB
Write any table (A–F)	Central DB (+ local for main PC basic tables)
Read local-only tables (G)	Local SQLite via `local_config_conn`
Write local-only tables (G)	Local SQLite via `local_config_conn`

When central_db_enabled = false (classic mode)

Operation	Connection Used
All reads and writes	Local SQLite

---

8. Switch-Back Safety

If the central DB goes offline or the admin sets `enabled=0`:

1. **Main PC:** Has full basic data in local SQLite (dual-written). Only TRENDLOG_DATA_DETAIL is missing locally — acceptable because historical trend data is not critical for immediate operation.
2. **Reader PC:** Has its own panels' basic data locally. Can resume classic mode instantly.
3. **No data loss** for operational data — only historical trendlog detail is in central only.

---

9. Edge Cases

Scenario	Behavior
Central DB unreachable at startup	Fall back to classic mode, log warning
Central DB goes down mid-operation	Queue failed writes, retry with backoff, log errors
Two PCs claim "main" role	Both write to central — last-write-wins for basic data (safe for UPSERT operations). TRENDLOG_DATA_DETAIL uses INSERT so no conflict.
INI has role=main but no DB config	Treat as classic mode (enabled but not configured)
Reader PC has no network to central	Operates in classic mode with local SQLite
Schema version mismatch	Run migration on connect, compare `database.version` in APPLICATION_CONFIG

---

10. Implementation Phases

Phase 8: INI Configuration Reader

- Create `api/src/ini_config.rs` — parse `setting.ini` `[CentralDatabase]` section
- Wire into startup in `app_state.rs`

Phase 9: SYSTEM_LOGS Table

- Add `SYSTEM_LOGS` CREATE TABLE to all 4 SQL dialect files
- Create SeaORM entity and basic CRUD endpoint

Phase 10: Startup Wiring

- `create_t3_app_state()` reads INI → reads DB_BACKEND_CONFIG → sets T3AppState fields
- Connect to central DB, run migration
- Keep `local_config_conn` always pointing to local SQLite

Phase 11: FFI Dual-Write (Main PC)

- Modify `t3_ffi_sync_service.rs`: basic data → write to both connections
- TRENDLOG_DATA_DETAIL → write to central only
- Reader PC: FFI writes local only (no change from current behavior)

Phase 12: Web Server Routing

- All route handlers check `central_db_enabled`
- If enabled: read/write from `t3_device_conn` (central)
- Local-only tables always use `local_config_conn`
- Write operations on main PC also dual-write to local

Phase 13: INI API Endpoints

- `GET /api/database/backend/ini` — read current INI config
- `POST /api/database/backend/ini` — update INI config (local access only)

Phase 14: Frontend Updates

- Add role selector (main/reader) to DatabaseConfigPage
- Add store_logs toggle
- Show current mode status
- Disable role change for remote connections (security)

Phase 15: Testing & Verification

- Unit tests for INI parsing
- Integration tests for dual-write
- Manual test: two PCs, one main, one reader, verify data flows correctly